

RESUMO TÉCNICO — PROGRAMAÇÃO ORIENTADA A OBJETOS

Sistema Acadêmico Integrado — PlataformaEnsino

Projeto Integrado Multidisciplinar III · Universidade Paulista — UNIP · 2025

Nota: este documento descreve a estrutura que **será implementada** na próxima sprint do projeto. Os conceitos de Orientação a Objetos aqui detalhados — abstração, herança, polimorfismo e encapsulamento — representam decisões de design do domínio a serem codificadas em C# com ASP.NET Core e Entity Framework Core. Nenhuma funcionalidade descrita encontra-se ainda em produção.

1 ABSTRAÇÃO

A classe `Usuario` será declarada como **abstract** em C#, o que impedirá que objetos do tipo base sejam instanciados diretamente. Essa decisão de design garantirá que todo ator do sistema possua um perfil definido — `Aluno`, `Professor`, `Coordenador` ou `Admin` —, centraliza a lógica compartilhada em um único ponto e elimina duplicação de código nas subclasses.

No banco de dados, a hierarquia será mapeada pelo Entity Framework Core com a estratégia **Table Per Hierarchy (TPH)**: todos os perfis serão armazenados em uma única tabela `Usuarios`, discriminada pela coluna `TipoUsuario`, dispensando JOINS entre tabelas para recuperar qualquer perfil de usuário.

Tabela 1 – Atributos e métodos da classe abstrata `Usuario`

Atributo / Método	Tipo	Modificador	Descrição
<code>Id</code>	<code>int</code>	PK	Chave primária gerada pelo banco
<code>Nome</code>	<code>string(150)</code>	required	Nome completo do usuário
<code>Email</code>	<code>string</code>	required	Endereço de e-mail único na plataforma
<code>Cpf</code>	<code>string(11)</code>	required	CPF com exatamente 11 dígitos, sem formatação
<code>Telefone</code>	<code>string(20)</code>	required	Telefone de contato
<code>Cep / Endereço</code>	<code>string</code>	required	CEP, Rua, Número, Bairro, Cidade e Estado
<code>TipoUsuario</code>	<code>string</code>	private set	Perfil do usuário — definido em <code>ConfigurarAcesso()</code> ; incluído como claim no JWT
<code>SenhaHash</code>	<code>string</code>	private set	Hash BCrypt — alterável apenas por <code>ConfigurarAcesso()</code> ou <code>AtualizarSenhaHash()</code>
<code>Ativo</code>	<code>bool</code>	private set	Alternado exclusivamente por <code>Ativar()</code> e <code>Desativar()</code>
<code>DataCadastro</code>	<code>DateTime</code>	private set	Atribuído automaticamente na criação — imutável após isso
<code>ExibirDados()</code>	<code>string</code>	virtual	Retorna dados formatados do usuário — será sobrescrito por cada subclasse
<code>ConfigurarAcesso()</code>	<code>void</code>	encapsulado	Define <code>TipoUsuario</code> , <code>SenhaHash</code> e <code>Ativo</code> com validação
<code>AlterarDados()</code>	<code>void</code>	encapsulado	Atualiza dados pessoais e de endereço de forma controlada

Fonte: modelagem do domínio — PlataformaEnsino (sprint futura).

2 HERANÇA

Quatro classes concretas herdarão de `Usuario`, cada uma representando um perfil distinto da plataforma. A herança permitirá que as subclasses reaproveitem todos os atributos e comportamentos da classe pai e acrescentem somente os elementos específicos de cada papel.

2.1 Aluno

Representará o estudante matriculado em cursos da plataforma. Além dos atributos herdados de `Usuario`, possuirá `Matricula` (código da matrícula institucional), `TurmaAtual` (turma em que o aluno está alocado) e a coleção `Matriculas` (lista de matrículas em cursos, associação 1:N). Sobrescreverá `ExibirDados()` acrescentando perfil, número de matrícula e turma atual.

2.2 Professor

Representará o docente responsável pela criação de conteúdo didático e avaliações. Incluirá os atributos `Especialidade` (área acadêmica de atuação) e `CodigoRegistro` (registro profissional), além das coleções `QuestoesBanco`, `ConteudosDidaticos` e `AvaliacoesPublicadas`. Sobrescreverá `ExibirDados()` acrescentando perfil e especialidade.

2.3 Coordenador

Representará o gestor acadêmico de um curso. Possuirá `CodigoRegistro` e `CursoResponsavel` (campo nullable que indica o curso sob sua responsabilidade). Sobrescreverá `ExibirDados()` acrescentando perfil e curso responsável.

2.4 Admin

Representará o administrador da plataforma com acesso irrestrito. Não possuirá atributos exclusivos — herdará integralmente os atributos de `Usuario`. Sobrescreverá `ExibirDados()` indicando o nível de acesso total ao sistema.

Tabela 2 – Visão geral da hierarquia de herança

Subclasse	Papel na Plataforma	Atributos Exclusivos	Associações Diretas
Aluno	Estudante matriculado	Matricula, TurmaAtual	List<Matricula>
Professor	Docente criador de conteúdo	Especialidade, CodigoRegistro	List<QuestaoBanco>, List<Avaliacao>, List<ConteudoDidatico>
Coordenador	Gestor acadêmico de curso	CursoResponsavel, CodigoRegistro	Curso (via CursoResponsavel)
Admin	Administrador da plataforma	—	List<Curso> (via CriadoPor)

Fonte: modelagem do domínio — PlataformaEnsino (sprint futura).

3 POLIMORFISMO

O método `ExibirDados()` será declarado **virtual** na classe `Usuario` e sobrescrito (**override**) em cada uma das quatro subclasses. O comportamento do método variará conforme o tipo concreto do objeto em tempo de execução, sem que o código chamador precise conhecer o tipo específico — característica central do polimorfismo em orientação a objetos.

Todas as implementações invocarão `base.ExibirDados()` antes de acrescentar suas informações específicas, demonstrando polimorfismo por extensão e não por substituição total do comportamento da classe pai.

Tabela 3 – Comportamento polimórfico de `ExibirDados()` por subclasse

Classe	Modificador	Informação Acrescentada ao Retorno da Base
Usuario	virtual	Nome, E-mail, CPF, Telefone, Endereço completo, Data de cadastro
Aluno	override	Perfil: ALUNO · Número de matrícula · Turma atual
Professor	override	Perfil: PROFESSOR · Especialidade acadêmica
Coordenador	override	Perfil: COORDENADOR · Curso responsável
Admin	override	Perfil: ADMINISTRADOR DO SISTEMA (Acesso Total)

Fonte: modelagem do domínio — PlataformaEnsino (sprint futura).

O polimorfismo também se manifestará na camada de autorização. O campo `TipoUsuario`, definido via `ConfigurarAcesso()`, será incluído como *claim* no token JWT emitido pelo endpoint `POST /api/auth/login`. Os controllers utilizarão o atributo `[Authorize(Roles = "...")]` para restringir endpoints com base no tipo concreto do usuário autenticado em tempo de execução.

Tabela 4 – Controle de acesso por perfil (polimorfismo de autorização)

TipoUsuario (claim JWT)	Prefixo de Rota	Escopo de Acesso
Admin	/api/admin/**	Gestão completa de usuários, cursos e configurações da plataforma
Coordenador	/api/coordenador/**	Gerenciamento de cursos, turmas, matrículas e professores vinculados
Professor	/api/professor/**	Criação de conteúdo, questões, avaliações e lançamento de notas
Aluno	/api/aluno/**	Acesso a conteúdos, submissão de avaliações e visualização de progresso

Fonte: modelagem do domínio — PlataformaEnsino (sprint futura).

4 ENCAPSULAMENTO

O encapsulamento será aplicado por meio de propriedades com modificador `private set`, impedindo que o estado interno dos objetos seja alterado diretamente por código externo à classe. Toda mutação de estado ocorrerá exclusivamente por meio de métodos de negócio controlados, que aplicarão as devidas validações antes de aceitar a alteração.

Tabela 5 – Propriedades encapsuladas e métodos de controle

Classe	Propriedade	Modificador	Método que controla a mutação
Usuario	SenhaHash	private set	ConfigurarAcesso() · AtualizarSenhaHash()
Usuario	TipoUsuario	private set	ConfigurarAcesso() — definido uma única vez
Usuario	Ativo	private set	Ativar() · Desativar()
Usuario	DataCadastro	private set	Preenchido na criação — imutável após isso
Matricula	Status	private set	Aprovar() · Rejeitar() · Cancelar()
Matricula	NotaFinal	private set	LancarNotaFinal() — valida se status é <i>Aprovada</i> e se nota está entre 0 e 10
Matricula	TurmaId	private set	VincularTurma() — valida integridade do identificador informado
Matricula	DataSolicitacao	private set	RegistrarSolicitacao() — imutável após o registro
Avaliacao	StatusPublicacao	private set	Publicar() · Arquivar() · VoltarParaRascunho()
Avaliacao	ProfessorAutorId	private set	DefinirAutor() — atribuído uma única vez com validação
Avaliacao	TentativasPermitidas	private set	DefinirTentativasPermitidas() — mínimo de 1 tentativa
Avaliacao	PublicadoEm	private set	Preenchido automaticamente ao invocar Publicar()

Fonte: modelagem do domínio — PlataformaEnsino (sprint futura).

5 ENUMERAÇÕES DE DOMÍNIO

As enumerações serão centralizadas no arquivo `PedagogicalEnums.cs` e definirão os estados válidos de cada entidade do domínio pedagógico, impedindo valores arbitrários em campos de estado e tornando o código autodocumentado.

StatusMatricula		TipoConteudoDidatico		TipoQuestao	
#	Valor	#	Valor	#	Valor
1	Pendente	1	Texto	1	Múltipla Escolha
2	Aprovada	2	PDF	2	Verdadeiro / Falso
3	Rejeitada	3	Vídeo	3	Discursiva
4	Cancelada	4	Link		

TipoAvaliacao		StatusPublicacao		StatusTentativaAvaliacao	
#	Valor	#	Valor	#	Valor
1	Quiz	1	Rascunho	1	Em Andamento
2	Prova	2	Publicado	2	Enviada
3	Exercício	3	Arquivado	3	Corrigida
				4	Expirada

StatusProgressoAprendizagem		OrigemCorrecaoNota		EscopoMarcoProgresso	
#	Valor	#	Valor	#	Valor
1	Não Iniciado	1	Automática	1	Curso
2	Em Andamento	2	Manual	2	Módulo
3	Concluído	3	Mista		